

Fundamentals of Deep Learning and Programming

Lecture 10: Transformer (2)

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Term Project

Due date: Two weeks later

Week 15 (12.12, Friday)

Should **submit** an **5 ~ 6 page report** as a term-project.

December						
Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Final Exam

Week 16 (12.19, Friday)

Final exam covers **all the material (=everything)** learned in class, including the midterm exam.

torch.Tensor.masked_fill() in PyTorch

```
import torch

x = torch.tensor([1.0, 2.0, 3.0, 4.0])

# Create a boolean mask
mask = torch.tensor([False, True, False, True])

# Use masked_fill
y = x.masked_fill(mask, -5.0) # elements with True in the mask are replaced by -5.0

print("Original tensor:", x)
print("Mask:", mask)
print("After masked_fill:", y)
```

```
Original tensor: tensor([1., 2., 3., 4.])
Mask: tensor([False,  True, False,  True])
After masked_fill: tensor([ 1., -5.,  3., -5.])
```


torch.Tensor.masked_fill() in PyTorch

```
import torch

# Example score tensor (e.g., attention scores)
scores = torch.tensor([1.0, 2.0, 3.0, 4.0])

# Mask: 1 = keep, 0 = mask out
mask = torch.tensor([1, 0, 1, 0])

# Apply masked_fill: mask==0 → set score to -inf
masked_scores = scores.masked_fill(mask == 0, float("-inf"))

print("Original scores:", scores)
print("Mask:", mask)
print("Masked scores:", masked_scores)

# Apply softmax to see the effect
attention = torch.softmax(masked_scores, dim=0)
print("Attention weights:", attention)
```

maks_fill_1.py

```
Original scores: tensor([1., 2., 3., 4.])
Mask: tensor([1, 0, 1, 0])
Masked scores: tensor([1., -inf, 3., -inf])
Attention weights: tensor([0.1192, 0.0000, 0.8808, 0.0000])
```

- `masked_fill(condition, value)` replaces all elements of scores where condition is True with the given value.

```
masked_scores = scores.masked_fill(mask == 0, float("-inf"))
```

- Wherever `mask == 0` → **replace** the corresponding scores element with $-\infty$.
- Wherever `mask == 1` → keep the original scores value.

Setting values to $-\infty$ (`"-inf"`) is commonly used in attention mechanisms and softmax:

- Softmax of $-\infty$ becomes 0, so these positions will not receive any attention weight.
- This is how we mask out padded tokens or prevent attention to certain positions.

Although the mathematical form is commonly written as:

$$y = Wx + b,$$

PyTorch implements the computation as:

$$y = xW^T + b,$$

which is equivalent but adapted for efficient batch computation and memory layout.

nn.Linear in PyTorch

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=4, out_features=3)

# y = x @ layer.weight.T + layer.bias
# x: input
# y: output
print(f"layer.weight : {layer.weight}") # weight shape = (3, 4)
print(f"layer.bias   : {layer.bias}") # bias shape = (3,)
```

PyTorch performs the linear transformation as:

$$y = xW^T + b.$$

This ensures that:

$$x \in \mathbb{R}^{(B \times I)}, \quad W^T \in \mathbb{R}^{(I \times O)} \quad \Rightarrow \quad xW^T \in \mathbb{R}^{(B \times O)},$$

which matches the expected output shape.

Save and Load Models using PyTorch

```
import torch
import torch.nn as nn

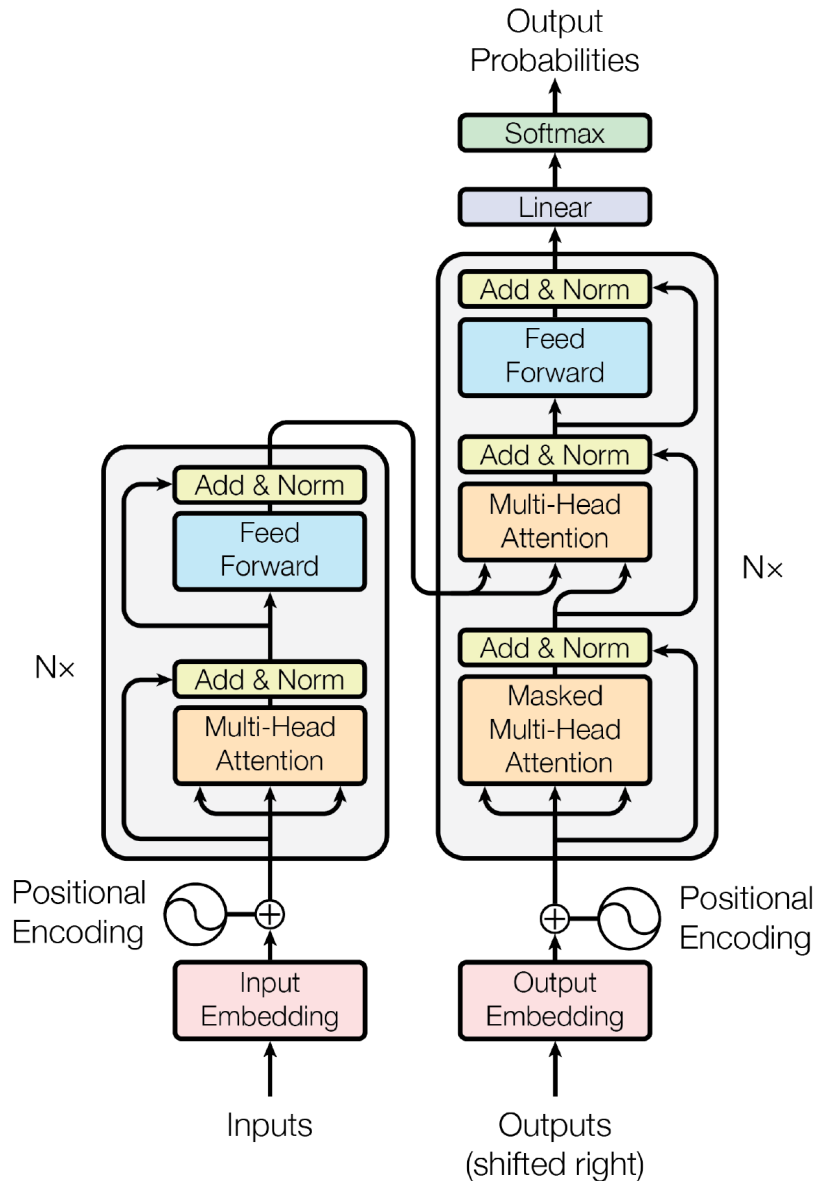
# -----
# Simple model definition
# -----
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(4, 3)

    def forward(self, x):
        return self.linear(x)

# -----
# 1. Create a model and inspect initial weights
# -----
model = SimpleModel()
print("Initial model weights:")
print(model.linear.weight)
print()
```

```
Initial model weights:
Parameter containing:
tensor([[ 0.2515, -0.3357,  0.1317,  0.3983],
        [-0.4345,  0.2308, -0.3156,  0.4657],
        [ 0.0736, -0.1300,  0.4589, -0.1545]], requires_grad=True)
```

Transformer model architecture



"Attention is All you Need" (<https://arxiv.org/abs/1706.03762>)
NIPS 2017

Why Transformer use Positional Encoding ?

Because Transformers do not process tokens sequentially like RNNs, they need a way to represent the order of tokens. They add **positional encodings** P to the input embeddings:

$$Z = X + P.$$

The original paper uses sinusoidal functions for positional encodings, so that the model can infer relative positions.

Positional encoding is designed to **inject** information about the **sequence position** into token embeddings.

Smooth and Continuous Encoding

The sinusoidal positional encoding uses a combination of sine and cosine functions of varying frequencies. For a token at position pos within the sequence and for dimension i of the embedding vector, the encoding is defined as:

$$\begin{aligned} \text{PE}(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \\ \text{PE}(pos, 2i + 1) &= \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

where:

- pos is the position of the token (from 0 to $\text{max_len} - 1$).
- i is the dimension index (from 0 to $d_{\text{model}}/2 - 1$).
- d_{model} is the dimensionality of the embedding.

Because sine and cosine are smooth and continuous, nearby positions have similar encodings. High-frequency components capture short-range patterns, while low-frequency components capture long-range dependencies.

Understanding Sinusoidal Positional Encoding

A key identity used in understanding sinusoidal positional encoding is:

$$\cos(a - b) = \cos a \cos b + \sin a \sin b,$$

This identity shows that $\cos(a - b)$ can be computed using only sine and cosine values of a and b .

We can define the positional encoding as follows: For a frequency ω and position pos :

$$PE(pos) = \begin{bmatrix} \cos(\omega \cdot pos) \\ \sin(\omega \cdot pos) \end{bmatrix}.$$

Thus, two positions pos_1 and pos_2 are encoded as:

$$PE(pos_1) = \begin{bmatrix} \cos(\omega pos_1) \\ \sin(\omega pos_1) \end{bmatrix}, \quad PE(pos_2) = \begin{bmatrix} \cos(\omega pos_2) \\ \sin(\omega pos_2) \end{bmatrix}.$$

Why This Helps Self-Attention ?

Self-attention computes similarity scores using dot products:

$$\text{score}(i, j) = Q_i \cdot K_j.$$

When positional encodings are added to token embeddings, the sinusoidal structure ensures that the dot products between queries and keys contain information about:

$$pos_i - pos_j,$$

allowing the model to infer how far apart two positions are.

This happens automatically due to the mathematical properties of sine and cosine.

- The dot product between encodings depends only on relative position differences.
- Self-attention readily exploits this because it is based on dot products.

Dot Product Encodes Relative Distance

Compute the dot product:

$$PE(pos_1) \cdot PE(pos_2) = \cos(\omega pos_1) \cos(\omega pos_2) + \sin(\omega pos_1) \sin(\omega pos_2).$$

Using the identity:

$$\cos(x - y) = \cos x \cos y + \sin x \sin y,$$

we obtain:

$$PE(pos_1) \cdot PE(pos_2) = \cos(\omega(pos_1 - pos_2)).$$

Important Observation: The dot product depends only on $(pos_1 - pos_2)$, meaning that sinusoidal encodings naturally encode **relative position information**.

Understanding Sinusoidal Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$
$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

where:

- pos is the position of the token (from 0 to $\text{max_len} - 1$).
- i is the dimension index (from 0 to $d_{\text{model}}/2 - 1$).
- d_{model} is the dimensionality of the embedding.

The term $10000^{2i/d_{\text{model}}}$ is equivalent to $\exp(\frac{2i}{d_{\text{model}}} \cdot \ln(10000))$, which is implemented in the code as $\exp(\frac{2i}{d_{\text{model}}} \cdot \ln(10000.0))$. The code computes the reciprocal of the denominator's base:

$$\text{div_term} = \exp\left(\frac{2i}{d_{\text{model}}} \cdot (-\ln(10000.0))\right) = \frac{1}{10000^{2i/d_{\text{model}}}}$$

This `div_term` is then multiplied by the position vector, effectively computing the argument of the sine and cosine functions.

```
# frequency term for even dimensions
div_term = torch.exp(
    torch.arange(0, d_model, 2, dtype=torch.float32)
    * (-math.log(10000.0) / d_model)
)
```

Positional Encoding (1) : separate heatmaps for sin and cos

```
import torch
import math
import matplotlib.pyplot as plt

# =====
# Generate sinusoidal positional encoding
# =====

def generate_positional_encoding(d_model, max_len=200):

    # pe shape: (max_len, d_model)
    pe = torch.zeros(max_len, d_model)

    # positions: 0 .. max_len-1, shape = (max_len, 1)
    position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)

    # frequency term for even dimensions
    div_term = torch.exp(
        torch.arange(0, d_model, 2, dtype=torch.float32)
        * (-math.log(10000.0) / d_model)
    )

    # Even indices: sin, shape broadcasting: (max_len, 1) * (1, d_model/2)
    pe[:, 0::2] = torch.sin(position * div_term)
    # Odd indices: cos
    pe[:, 1::2] = torch.cos(position * div_term)

    return pe # (max_len, d_model)
```

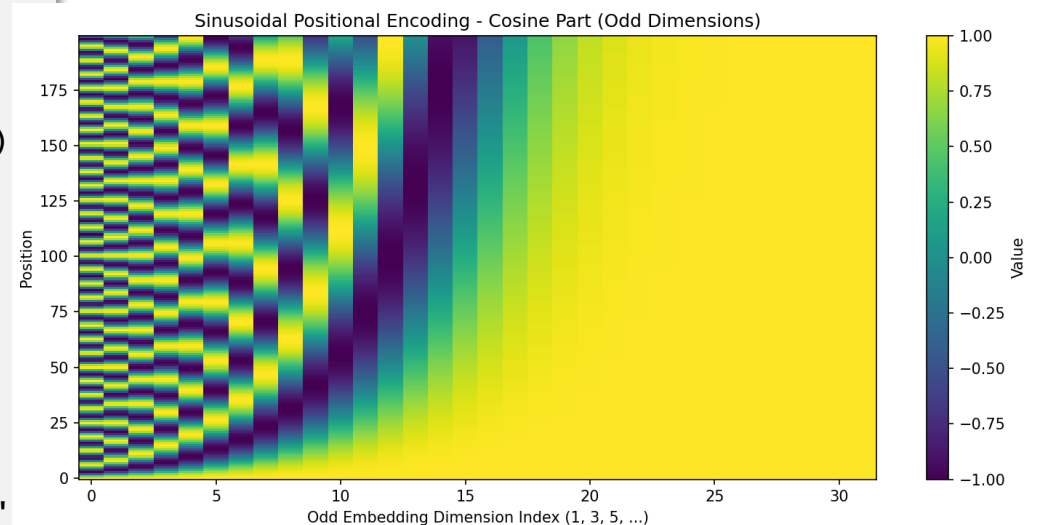
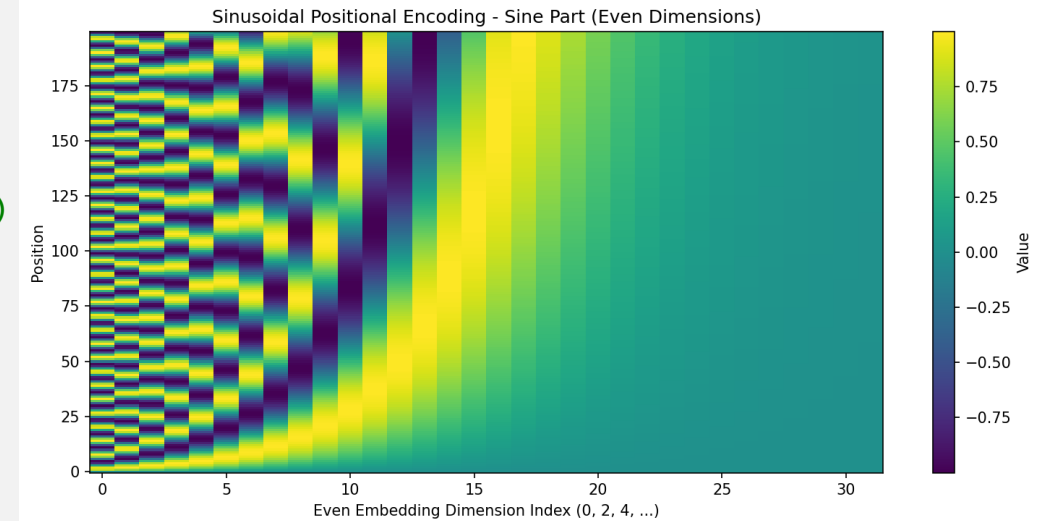
```
# =====
# Visualization: separate heatmaps for sin and cos
# =====

d_model = 64 # embedding dimension
max_len = 200 # number of positions
pe = generate_positional_encoding(d_model, max_len) # (max_len, d_model)

# Extract even (sin) and odd (cos) dimensions
# sin_part: shape (max_len, d_model/2)
# cos_part: shape (max_len, d_model/2)
sin_part = pe[:, 0::2]
cos_part = pe[:, 1::2]

# ----- Heatmap 1: Sine components (even dimensions) -----
plt.figure(figsize=(10, 5))
plt.imshow(sin_part, aspect="auto", origin="lower", cmap="viridis")
plt.colorbar(label="Value")
plt.xlabel("Even Embedding Dimension Index (0, 2, 4, ...)")
plt.ylabel("Position")
plt.title("Sinusoidal Positional Encoding - Sine Part (Even Dimensions)")
plt.tight_layout()

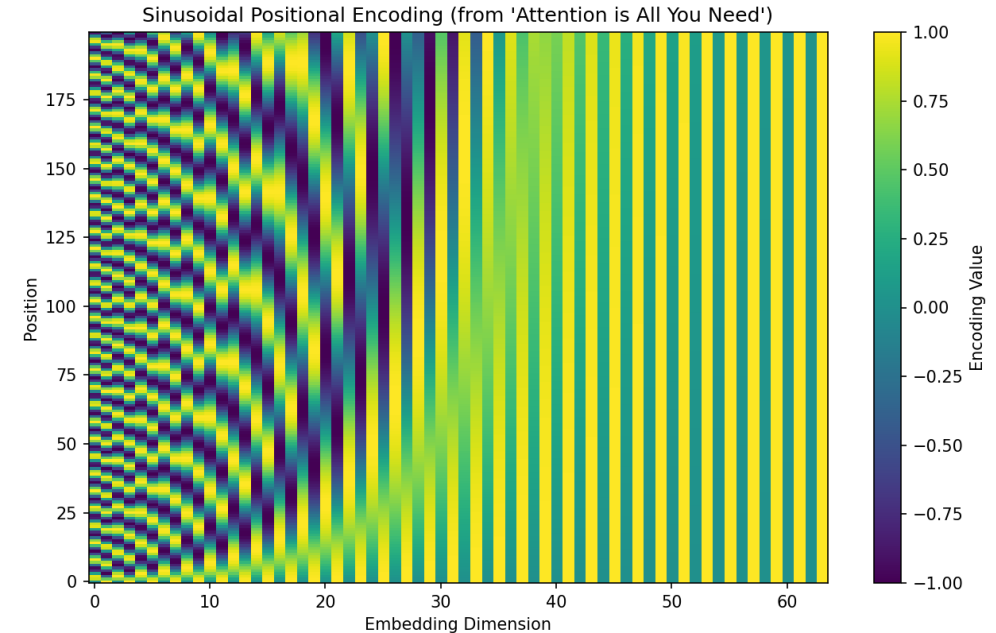
# ----- Heatmap 2: Cosine components (odd dimensions) -----
plt.figure(figsize=(10, 5))
plt.imshow(cos_part, aspect="auto", origin="lower", cmap="viridis")
plt.colorbar(label="Value")
plt.xlabel("Odd Embedding Dimension Index (1, 3, 5, ...)")
plt.ylabel("Position")
plt.title("Sinusoidal Positional Encoding - Cosine Part (Odd Dimensions)")
plt.tight_layout()
plt.show()
```



Positional Encoding (2) : heatmap that combines sin and cos

```
# =====  
# Visualization  
# =====  
  
d_model = 64 # number of embedding dimensions (x-axis)  
max_len = 200 # number of positions (y-axis)  
  
pe = generate_positional_encoding(d_model, max_len) # shape = (200, 64)  
  
plt.figure(figsize=(10, 6))  
  
# imshow expects (rows, cols) = (positions, dimensions)  
plt.imshow(pe, cmap="viridis", aspect="auto", origin="lower")  
  
plt.xlabel("Embedding Dimension")  
plt.ylabel("Position")  
plt.title("Sinusoidal Positional Encoding (from 'Attention is All You Need')")  
plt.colorbar(label="Encoding Value")  
  
plt.show()
```

pe_1.py



```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=5000):
        super().__init__()

        # pe: (max_len, d_model)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2, dtype=torch.float32)
            * (-math.log(10000.0) / d_model)
        )
        # Even indices: sin
        pe[:, 0::2] = torch.sin(position * div_term)
        # Odd indices: cos
        pe[:, 1::2] = torch.cos(position * div_term)

        # Register as buffer so it is saved with the model but not a learnable parameter
        self.register_buffer("pe", pe.unsqueeze(0)) # (1, max_len, d_model)

    def forward(self, x):
        # x: (batch_size, seq_len, d_model)
        seq_len = x.size(1)

        # Add positional encoding (broadcast along batch dimension)
        x = x + self.pe[:, :seq_len, :]
        return x
```

The **scaled dot-product attention** computes a weighted sum of the value vectors. For a set of queries Q , keys K , and values V , attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Explanation:

- $QK^\top$ computes all pairwise dot products between queries and keys.
- The division by $\sqrt{d_k}$ is the *scaling*; it keeps the magnitude of the dot products from growing too large when d_k is large.
- softmax turns each row into a probability distribution over all positions.
- Multiplying by V computes a weighted sum of value vectors for each query.

This allows each token to attend to (i.e. focus on) other tokens in the sequence.


```
def scaled_dot_product_attention(q, k, v, mask=None):  
  
    # q, k, v: (batch, heads, seq_len, head_dim) -> (B, H, L_q, D)  
    # mask: (batch, 1, 1, seq_len_k) or broadcastable shape (optional)  
  
    d_k = q.size(-1)  
  
    # q @ k.T = (B, H, L_q, D) @ (B, H, D, L_k) -> (B, H, L_q, L_k)  
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k) # (B, H, L_q, L_k)  
  
    if mask is not None:  
        # mask == 0 -> masked out  
        scores = scores.masked_fill(mask == 0, float("-inf"))  
  
    attn = F.softmax(scores, dim=-1)  
    out = torch.matmul(attn, v) # (B, H, L_q, head_dim)  
    return out, attn
```

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V.$$

Instead of using a single attention mechanism, Transformers use **multiple heads**.

Each head has its own projection matrices:

$$Q_h = XW_h^Q, \quad K_h = XW_h^K, \quad V_h = XW_h^V,$$

and produces:

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h).$$

Then all heads are concatenated:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

where W^O is another learned projection matrix.

Intuition:

- Each head can focus on different types of relationships (e.g. syntax, long-range dependencies).
- Concatenating them allows the model to combine these different perspectives.

```
class MultiHeadAttention(nn.Module):  
  
    # Multi-head attention as in the Transformer paper.  
  
    def __init__(self, d_model, num_heads, dropout=0.1):  
        super().__init__()  
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"  
  
        self.d_model = d_model  
        self.num_heads = num_heads  
        self.head_dim = d_model // num_heads  
  
        # Linear projections for q, k, v  
        self.w_q = nn.Linear(d_model, d_model)  
        self.w_k = nn.Linear(d_model, d_model)  
        self.w_v = nn.Linear(d_model, d_model)  
  
        # Final linear layer  
        self.w_o = nn.Linear(d_model, d_model)  
  
        self.dropout = nn.Dropout(dropout)
```

- `d_model`: the size of the input embedding (e.g., 512 or 256)
- `num_heads`: how many separate attention heads to split into
- `head_dim`: the dimension of each head
→ Multi-head attention splits the embedding into multiple smaller subspaces.

In the implementation of the Multi-Head Attention module, the layer `self.w_q` is defined as follows:

```
self.w_q = nn.Linear(d_model, d_model)
```

This layer plays a crucial role in generating the *query* vectors used in the attention mechanism.

The module `self.w_q` is a learnable linear transformation. It maps the input embedding of size d_{model} to another vector space of the same dimensionality.

$$\text{self.w_q}: \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}.$$

During training, this layer learns a weight matrix W_q and a bias vector b_q . When an input sequence $X \in \mathbb{R}^{B \times L \times d_{\text{model}}}$ is passed through the module, the query vectors are computed as:

$$Q = XW_q + b_q.$$

Why Do We Need Query Vectors?

In attention mechanisms, each token in the sequence is represented by three vectors:

- **Query (Q)**: Represents what the token is looking for.
- **Key (K)**: Represents what information the token offers.
- **Value (V)**: Represents the actual content used in attention.

The query vectors produced by `self.w_q` are used to compute similarity scores with the key vectors:

$$\text{score}(Q_i, K_j) = \frac{Q_i K_j^\top}{\sqrt{d_k}}.$$

These scores determine how much attention one token should pay to another.

```
def forward(self, q, k, v, mask=None):

    # q, k, v: (batch, seq_len, d_model)
    # mask: (batch, 1, 1, seq_len_k) or broadcastable

    B, L_q, _ = q.size()
    B, L_k, _ = k.size()
    B, L_v, _ = v.size()

    # Linear projections
    q = self.w_q(q) # (B, L_q, d_model)
    k = self.w_k(k) # (B, L_k, d_model)
    v = self.w_v(v) # (B, L_v, d_model)

    # Reshape to (B, H, L, head_dim)
    q = q.view(B, L_q, self.num_heads, self.head_dim).transpose(1, 2)
    k = k.view(B, L_k, self.num_heads, self.head_dim).transpose(1, 2)
    v = v.view(B, L_v, self.num_heads, self.head_dim).transpose(1, 2)

    # Apply scaled dot-product attention
    out, attn = scaled_dot_product_attention(q, k, v, mask=mask)

    # Concatenate heads
    out = out.transpose(1, 2).contiguous().view(B, L_q, self.d_model)

    # Final linear + dropout
    out = self.w_o(out)
    out = self.dropout(out)
    return out
```

```
# Linear projections  
q = self.w_q(q) # (B, L_q, d_model)
```

the object `self.w_q` is an instance of `nn.Linear`. PyTorch's `nn.Linear` is a *callable module*, meaning that the object behaves like a function. When you write `self.w_q(q)`, you are applying a linear transformation to the tensor `q`.

`nn.Linear` Is Callable

All subclasses of `nn.Module` in PyTorch override the `__call__` method. Thus, the expression

```
output = layer(input)  
is internally equivalent to  
output = layer.__call__(input)  
which then dispatches to  
layer.forward(input).
```

Therefore, using function-call syntax on the module is natural and expected.

```
# Linear projections  
q = self.w_q(q) # (B, L_q, d_model)
```

`self.w_q` is an instance of `nn.Linear`, which is a callable class.

The Linear Projection

The operation performed by

```
q = self.w_q(q)
```

corresponds to applying the linear transformation

$$q' = qW_q^T + b_q,$$

where

- $q \in \mathbb{R}^{B \times L_q \times d_{\text{model}}}$,
- $W_q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$,
- $b_q \in \mathbb{R}^{d_{\text{model}}}$.

The output retains the same shape:

$$(B, L_q, d_{\text{model}}).$$

Role in Multi-Head Attention

Transformer-style multi-head attention requires three projections:

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v.$$

Each uses a separate `nn.Linear` module. Thus, the line

```
q = self.w_q(q)
```

creates the query representation Q from the input sequence.

In multi-head attention, an input tensor q of shape

$$(B, L_q, d_{\text{model}})$$

must be reshaped and re-arranged so that it can be processed as multiple attention heads.

```
# Reshape to (B, H, L, head_dim)
q = q.view(B, L_q, self.num_heads, self.head_dim).transpose(1, 2)
```

performs exactly this transformation.

Reshaping the Tensor

The model dimension satisfies

$$d_{\text{model}} = H \times \text{head_dim},$$

where H is the number of attention heads and head_dim is the dimensionality within each head.

The command

```
q.view(B, L_q, H, head_dim)
```

reshapes the tensor from

$$(B, L_q, d_{\text{model}})$$

into

$$(B, L_q, H, \text{head_dim}).$$

This separates the model dimension into multiple heads.

```
# Reshape to (B, H, L, head_dim)
q = q.view(B, L_q, self.num_heads, self.head_dim).transpose(1, 2)
```

Transposing Dimensions

The next operation is

```
transpose(1, 2),
```

which swaps the second and third dimensions. The shape becomes:

$$(B, H, L_q, \text{head_dim}).$$

This reorders the tensor so that the head dimension is exposed early, which is the format required by scaled dot-product attention.

Why This Layout Is Needed

Most attention implementations expect queries, keys, and values in the shape:

$$(B, H, L_q, \text{head_dim}),$$

because:

- Each head processes the sequence independently.
- Attention weights are computed per head.
- Batch and head dimensions are kept separate for efficiency.

The transpose ensures that mathematical operations across heads can be performed efficiently as parallel matrix multiplications.

The full transformation:

```
q = q.view(B, L_q, H, head_dim).transpose(1, 2)
```

performs two important steps:

1. **Reshape:**

$$(B, L_q, d_{\text{model}}) \rightarrow (B, L_q, H, \text{head_dim})$$

2. **Reorder dimensions:**

$$(B, L_q, H, \text{head_dim}) \rightarrow (B, H, L_q, \text{head_dim})$$

This creates the correct tensor layout for multi-head attention.

```
# Apply scaled dot-product attention
```

```
out, attn = scaled_dot_product_attention(q, k, v, mask=mask)
```

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{mask}\right) V,$$

where:

- Q is the query matrix,
- K is the key matrix,
- V is the value matrix,
- d_k is the dimensionality of each key vector.

For each attention head, the model determines:

- **Which tokens to focus on:**

This is controlled by the similarity scores

$$QK^\top.$$

- **How much information each token contributes:**

After scaling and masking, the softmax produces weights that form a weighted sum:

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Thus, attention selectively aggregates information from the input sequence according to learned relevance scores.

```
out = out.transpose(1, 2).contiguous().view(B, L_q, self.d_model)
```

1. **Transpose:**

$$(B, H, L_q, \text{head_dim}) \rightarrow (B, L_q, H, \text{head_dim})$$

2. **Make memory contiguous:** ensures compatibility with `view`.

3. **Reshape:**

$$(B, L_q, H \times \text{head_dim}) = (B, L_q, d_{\text{model}})$$

This recombines the outputs of all attention heads into a single vector per token, restoring the standard Transformer output shape.

Ensuring Memory Contiguity

PyTorch's `transpose` operation returns a tensor with the same values but a different memory layout. Some operations, such as `view`, require the tensor to be stored contiguously.

Thus, the call:

```
out.contiguous()
```

creates a contiguous copy of the tensor in memory while keeping the shape:

Attention Is All You Need

Ashish Vaswani*
 Google Brain
 avaswani@google.com

Noam Shazeer*
 Google Brain
 noam@google.com

Niki Parmar*
 Google Research
 nikip@google.com

Jakob Uszkoreit*
 Google Research
 usz@google.com

Llion Jones*
 Google Research
 llion@google.com

Aidan N. Gomez*[†]
 University of Toronto
 aidan@cs.toronto.edu

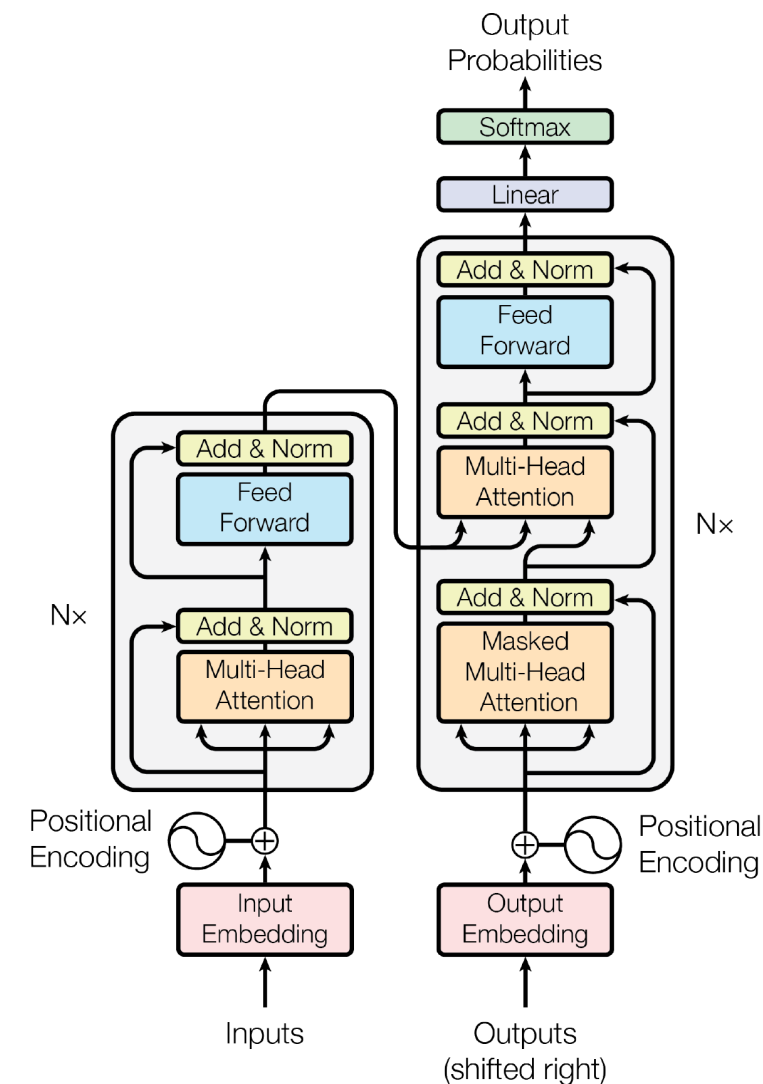
Lukasz Kaiser*
 Google Brain
 lukaszkaizer@google.com

Illia Polosukhin*[‡]
 illia.polosukhin@gmail.com

5.4 Regularization

We employ three types of regularization during training:

Residual Dropout We apply dropout [27] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.



Why Dropout Is Used Together with Layer Normalization in the Transformer

In the Transformer architecture, each encoder and decoder layer contains a sublayer structure of the form:

$$\text{LayerNorm}(x + \text{Dropout}(\text{Sublayer}(x))).$$

A natural question is why dropout is applied even though layer normalization is also present.

1. Purpose of Layer Normalization

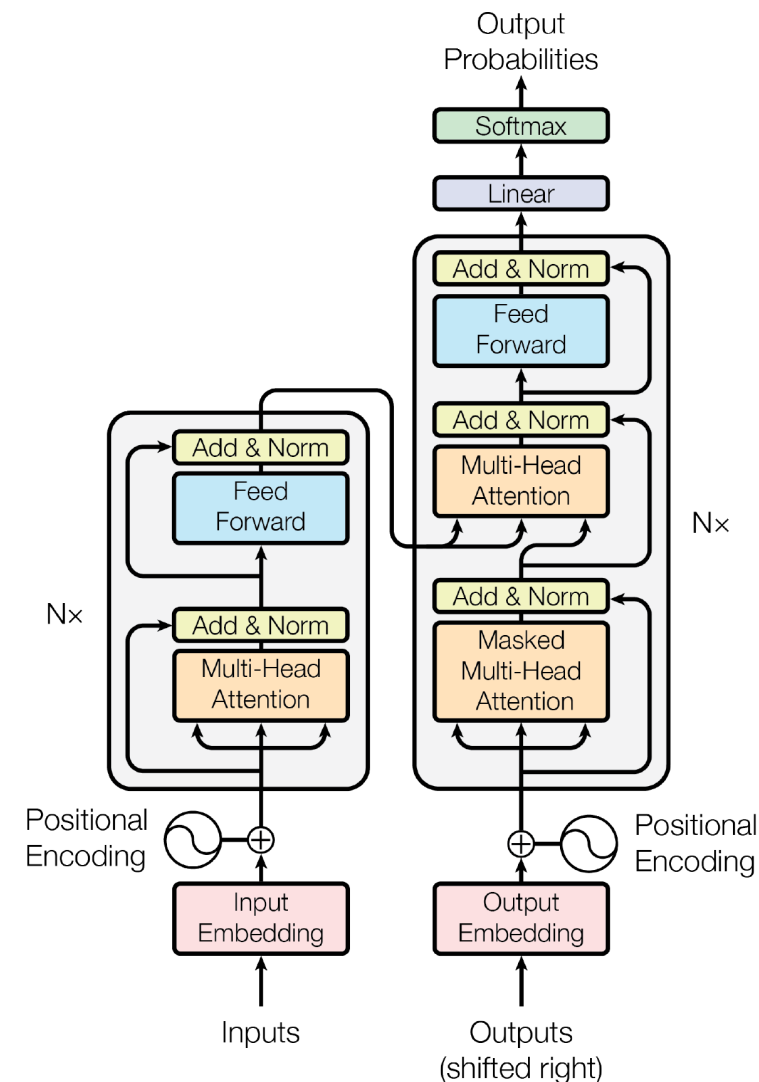
Layer normalization stabilizes training by normalizing activations across feature dimensions:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \gamma + \beta,$$

where μ and σ are the mean and standard deviation computed along the last dimension. Its main benefits are:

- reducing internal covariate shift,
- stabilizing gradients,
- allowing deeper and more stable training.

LayerNorm does *not* provide regularization and does not prevent overfitting.



Why Dropout Is Used Together with Layer Normalization in the Transformer

2. Purpose of Dropout

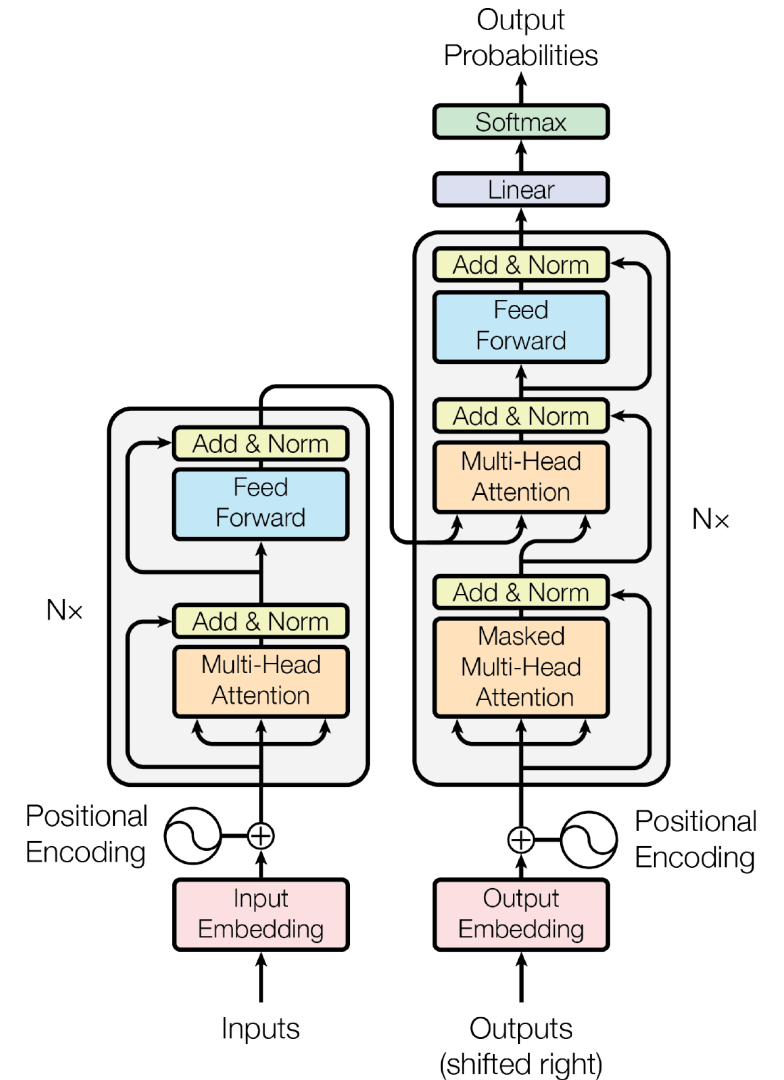
Dropout randomly removes units from the network during training:

$$x_{\text{drop}} = x \odot m, \quad m \sim \text{Bernoulli}(p),$$

which prevents:

- overfitting,
- co-adaptation of features,
- reliance on a small number of dominant dimensions.

Unlike LayerNorm, dropout directly regularizes the model. Therefore, normalization cannot replace dropout, and both tools play complementary roles.



Why Dropout Is Used Together with Layer Normalization in the Transformer

3. Why Both Are Used Together

Residual connections can cause models to propagate unregularized signals directly from earlier layers. To counteract this, the Transformer introduces dropout *before* the residual addition:

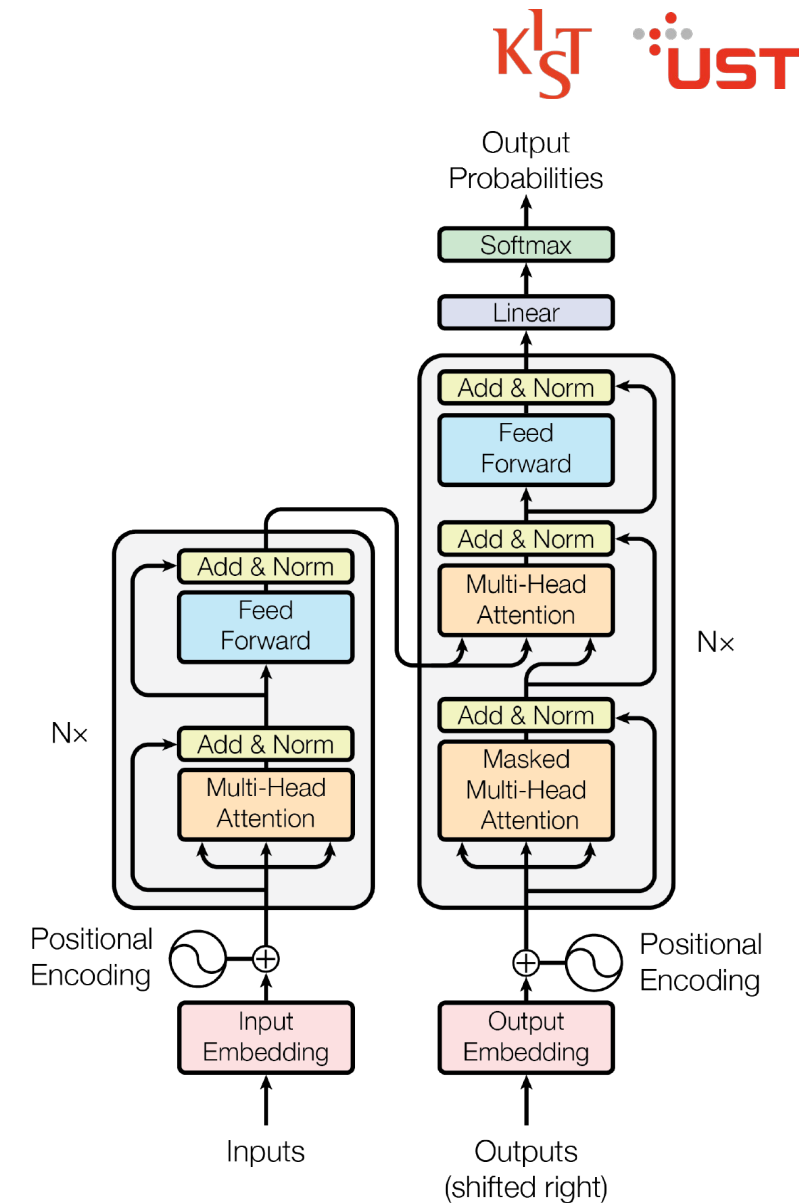
$$x + \text{Dropout}(\text{Sublayer}(x)).$$

After the residual connection, LayerNorm then stabilizes the combined signal:

$$\text{LayerNorm}(x + \text{Dropout}(\text{Sublayer}(x))).$$

Thus:

- Dropout provides regularization.
- LayerNorm stabilizes the resulting representation.
- Neither can replace the other.



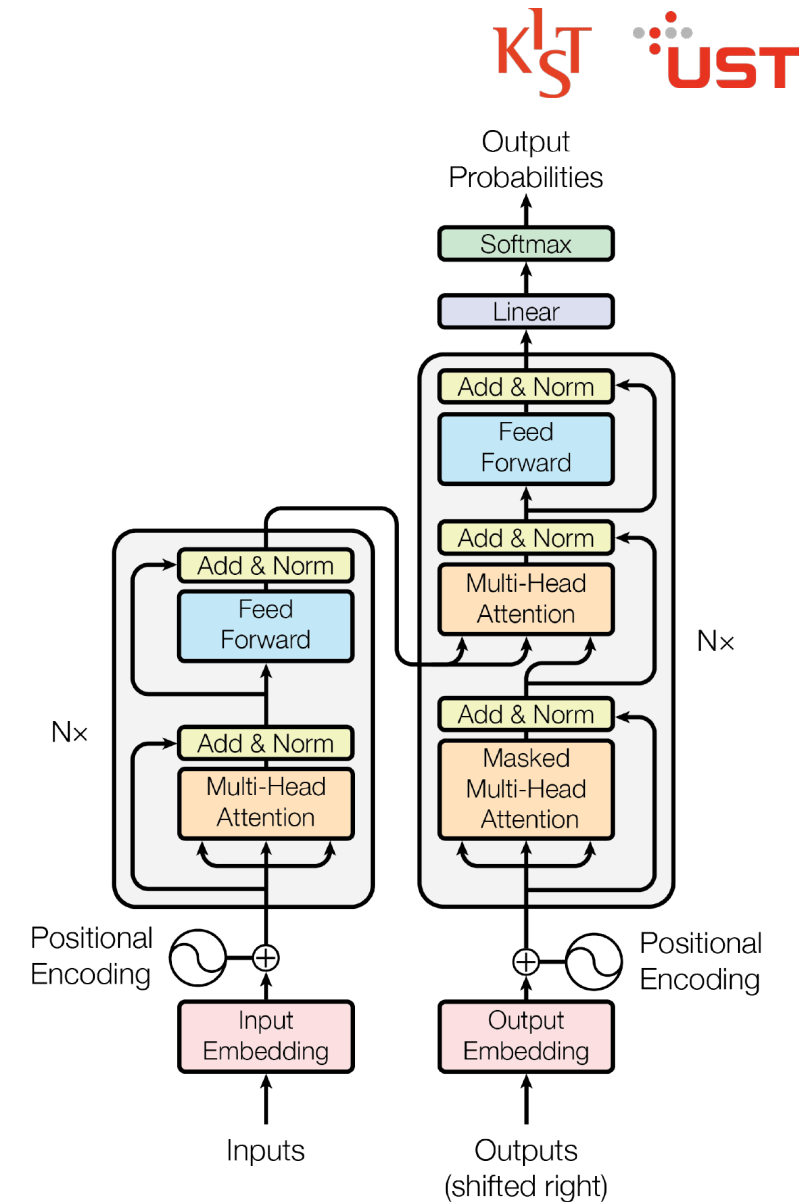
Why Dropout Is Used Together with Layer Normalization in the Transformer

Dropout and layer normalization serve fundamentally different functions in the Transformer:

- **LayerNorm** stabilizes and rescales activations.
- **Dropout** regularizes the model to reduce overfitting.

Layer Normalization, Batch Normalization, and Dropout address different aspects of deep learning models:

- LayerNorm and BatchNorm focus on stabilizing the optimization process, but along different axes (within a sample versus across a batch).
- Dropout focuses on improving generalization by explicitly regularizing the model.

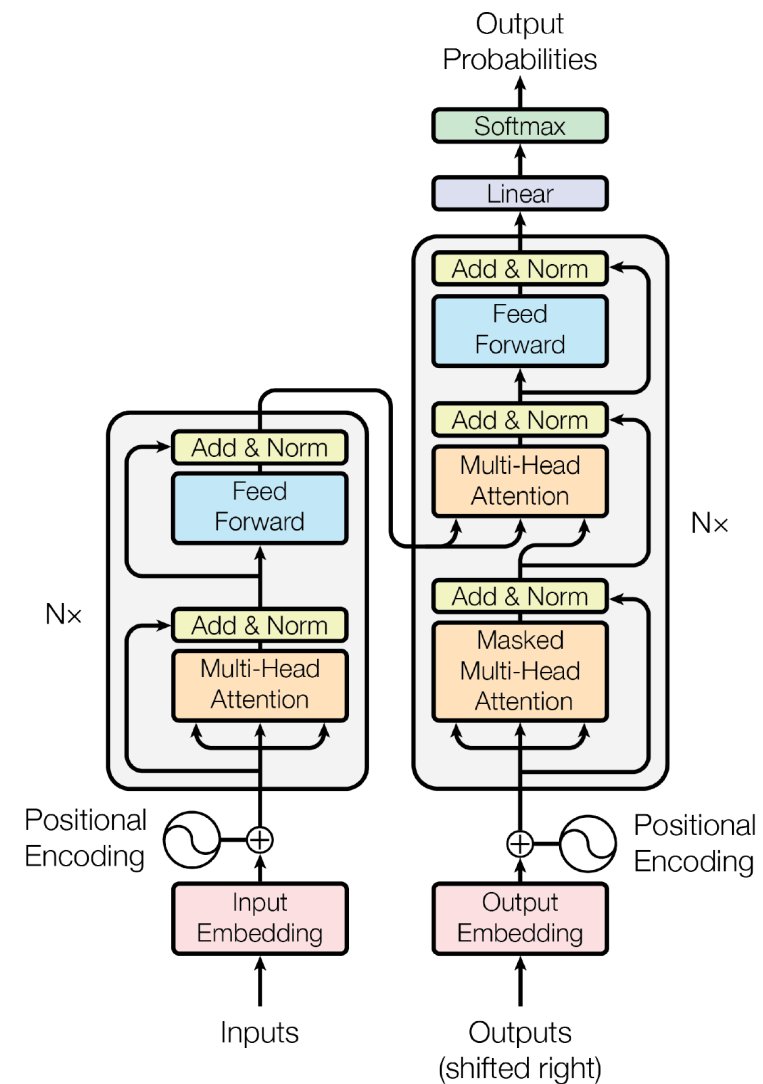


Position-wise Feed-Forward Network

[4]

```
class PositionwiseFeedForward(nn.Module):  
  
    # Two-layer feed-forward network applied at each position.  
    # FFN(x) = max(0, xW1 + b1)W2 + b2  
  
    def __init__(self, d_model, d_ff, dropout=0.1):  
        super().__init__()  
        self.fc1 = nn.Linear(d_model, d_ff)  
        self.fc2 = nn.Linear(d_ff, d_model)  
        self.dropout = nn.Dropout(dropout)  
  
    def forward(self, x):  
        x = self.fc1(x)  
        x = F.relu(x)  
        x = self.dropout(x)  
        x = self.fc2(x)  
        return x
```

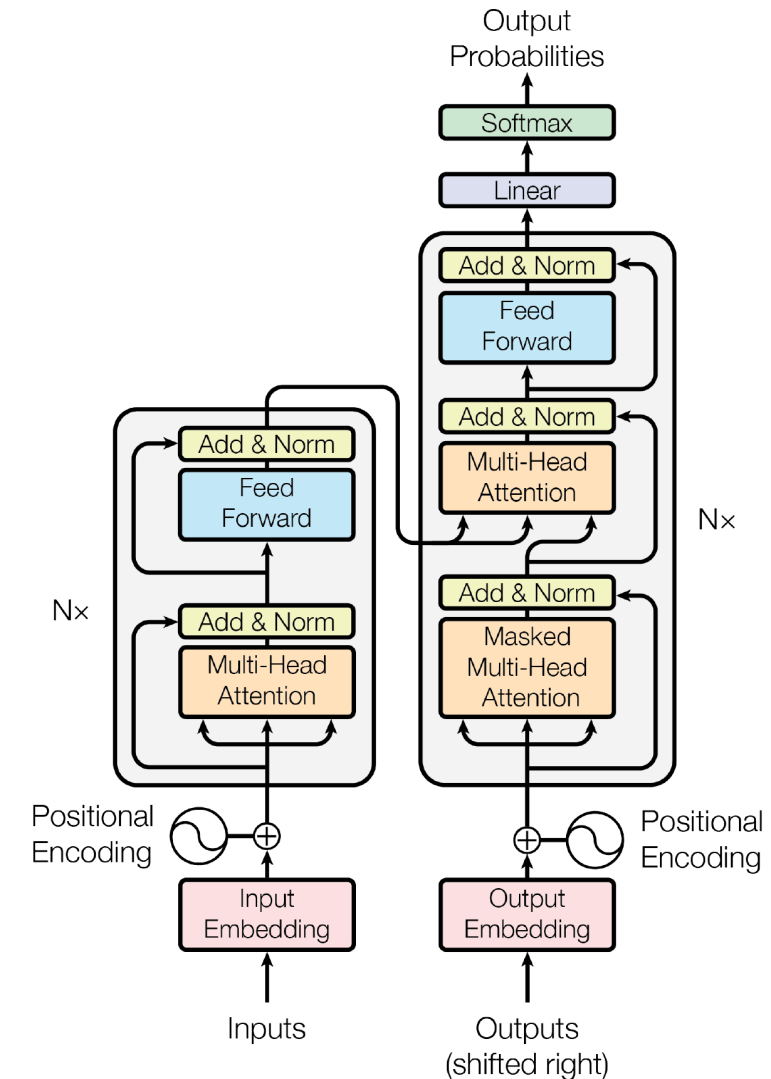
```
# Create model (smaller than paper for teaching / debugging)  
model = SimpleTransformer(  
    src_vocab_size=src_vocab_size,  
    tgt_vocab_size=tgt_vocab_size,  
    d_model=256,    # try 512 for paper-like  
    num_layers=2,   # try 6 for paper-like  
    num_heads=4,    # try 8 for paper-like  
    d_ff=512,       # try 2048 for paper-like  
)
```



Encoder and Decoder in Transformer

The original Transformer consists of:

- **Encoder:** a stack of identical layers, each containing multi-head self-attention and a position-wise feed-forward network.
- **Decoder:** a stack of layers that uses:
 - masked multi-head self-attention over the output sequence (to prevent looking into the future),
 - multi-head attention over the encoder outputs,
 - a feed-forward network.



```
class EncoderLayer(nn.Module):
```

```
    # One encoder layer = self-attention + feed-forward, with residual + layernorm.
```

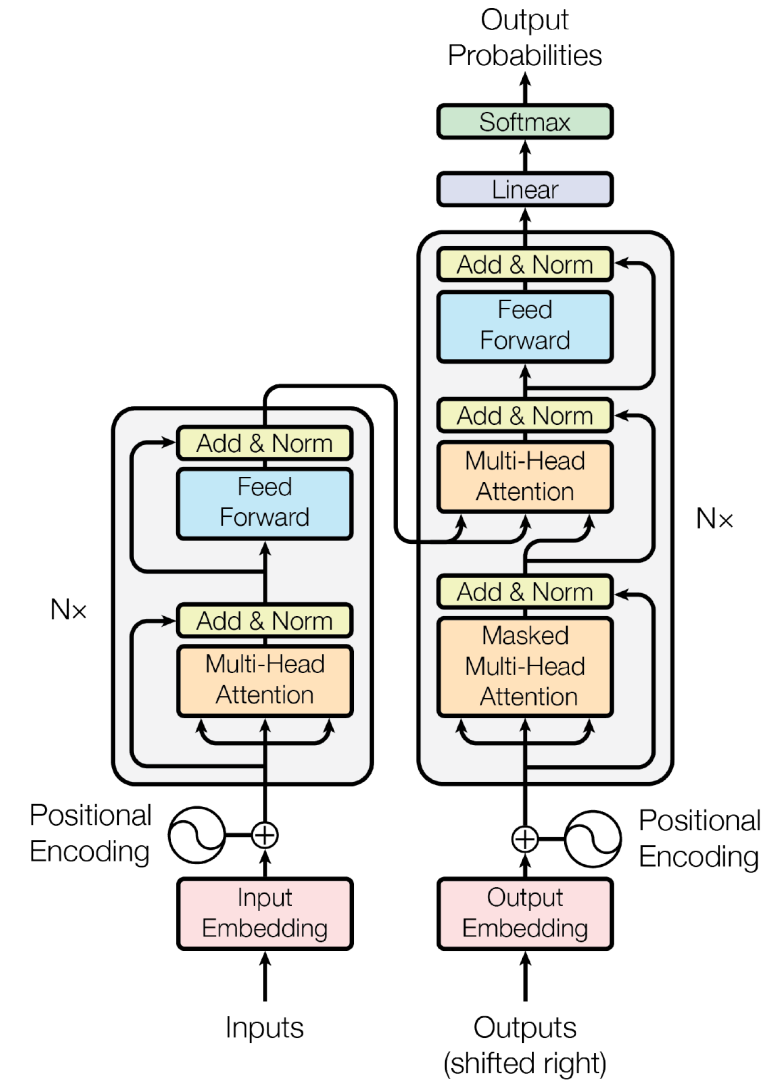
```
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ff = PositionwiseFeedForward(d_model, d_ff, dropout)
```

```
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, x, src_mask=None):
        # Self-attention sublayer
        attn_out = self.self_attn(x, x, x, mask=src_mask)
        x = x + self.dropout(attn_out)
        x = self.norm1(x)

        # Feed-forward sublayer
        ff_out = self.ff(x)
        x = x + self.dropout(ff_out)
        x = self.norm2(x)

        return x
```



In Transformer architectures, the terms *encoder layer* and *encoder* are closely related but refer to different levels of structure. Understanding the distinction between them is important for interpreting Transformer code implementations.

An **encoder layer** is a *single computational block* inside the Transformer encoder stack. A typical encoder layer contains:

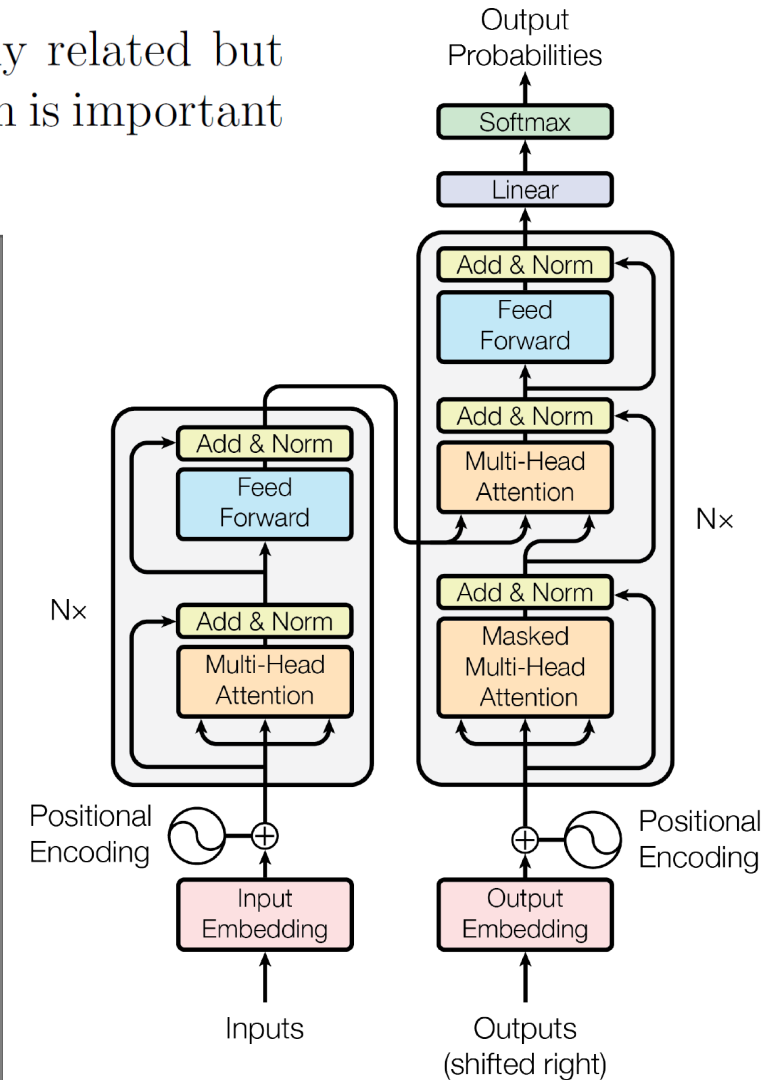
- Multi-Head Self-Attention module
- Add & Norm (residual connection + layer normalization)
- Position-wise Feed-Forward Network (FFN)
- Another Add & Norm step

Mathematically, for an input X , one encoder layer computes:

$$\tilde{X} = \text{LayerNorm}(X + \text{SelfAttention}(X))$$

$$\text{EncoderLayer}(X) = \text{LayerNorm}(\tilde{X} + \text{FFN}(\tilde{X})).$$

Thus, an encoder layer performs one round of self-attention and one feed-forward transformation. It does not process the entire model depth by itself.



In Transformer architectures, the terms *encoder layer* and *encoder* are closely related but refer to different levels of structure. Understanding the distinction between them is important for interpreting Transformer code implementations.

Encoder

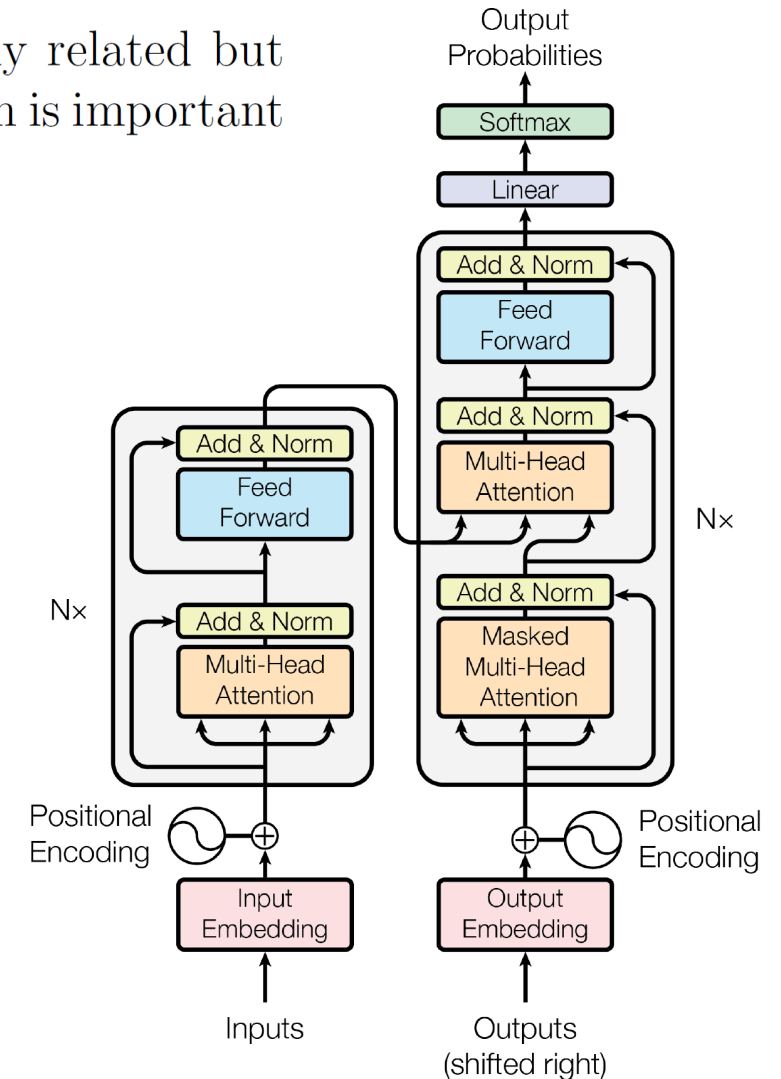
The **encoder** is the entire module composed of *multiple stacked encoder layers*. If the Transformer uses N layers, then the encoder applies:

$$X_0 = \text{Input Embeddings},$$

$$X_i = \text{EncoderLayer}_i(X_{i-1}) \quad \text{for } i = 1, 2, \dots, N,$$

$$\text{Encoder}(X_0) = X_N.$$

- **Granularity:** An encoder layer is a single module; the encoder is a stack of many such modules.
- **Depth:** The encoder layer implements one level of depth in the Transformer. The encoder implements all N levels.
- **Function:** The encoder layer performs one step of processing. The encoder combines all layers to produce the final encoded representation.
- **Analogy:** If each encoder layer is one “brick,” then the encoder is the entire “wall.”



```

class Encoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_layers, num_heads, d_ff,
                  max_len=5000, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos_enc = PositionalEncoding(d_model, max_len)

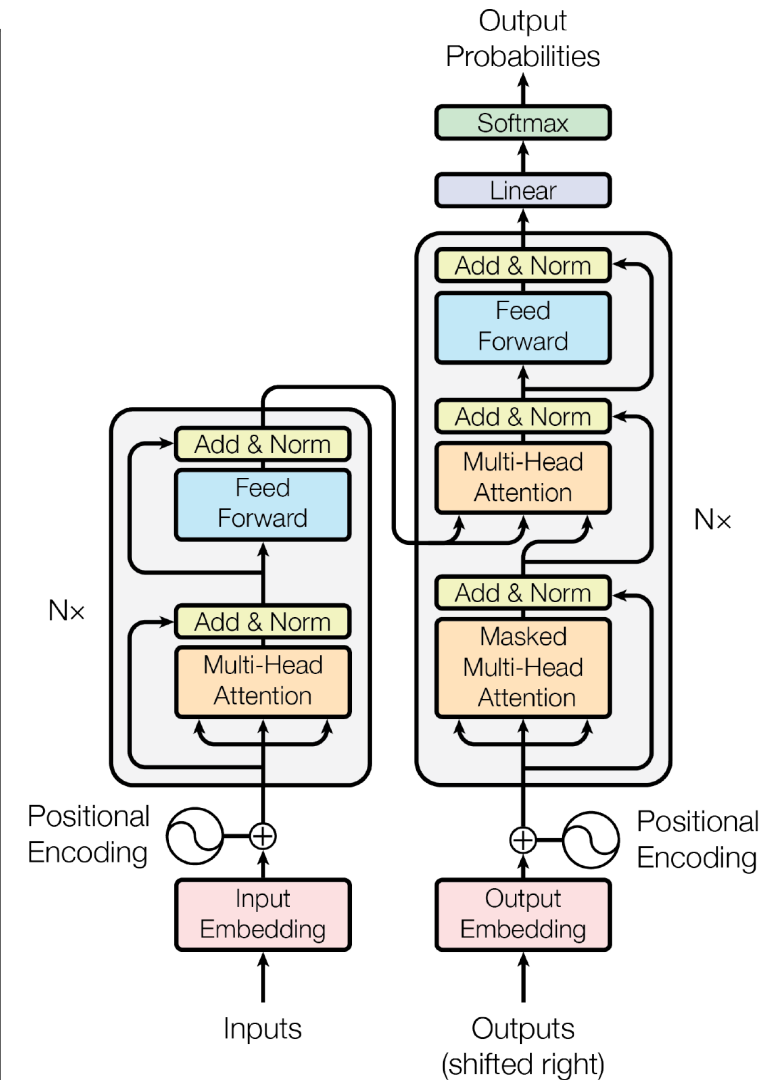
        self.layers = nn.ModuleList(
            [EncoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)]
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, src_mask=None):
        """
        src: (batch, src_len)
        src_mask: (batch, 1, 1, src_len) where 1=keep, 0=mask
        """
        x = self.embed(src) * math.sqrt(self.d_model)
        x = self.pos_enc(x)
        x = self.dropout(x)

        for layer in self.layers:
            x = layer(x, src_mask=src_mask)

        return x  # (batch, src_len, d_model)

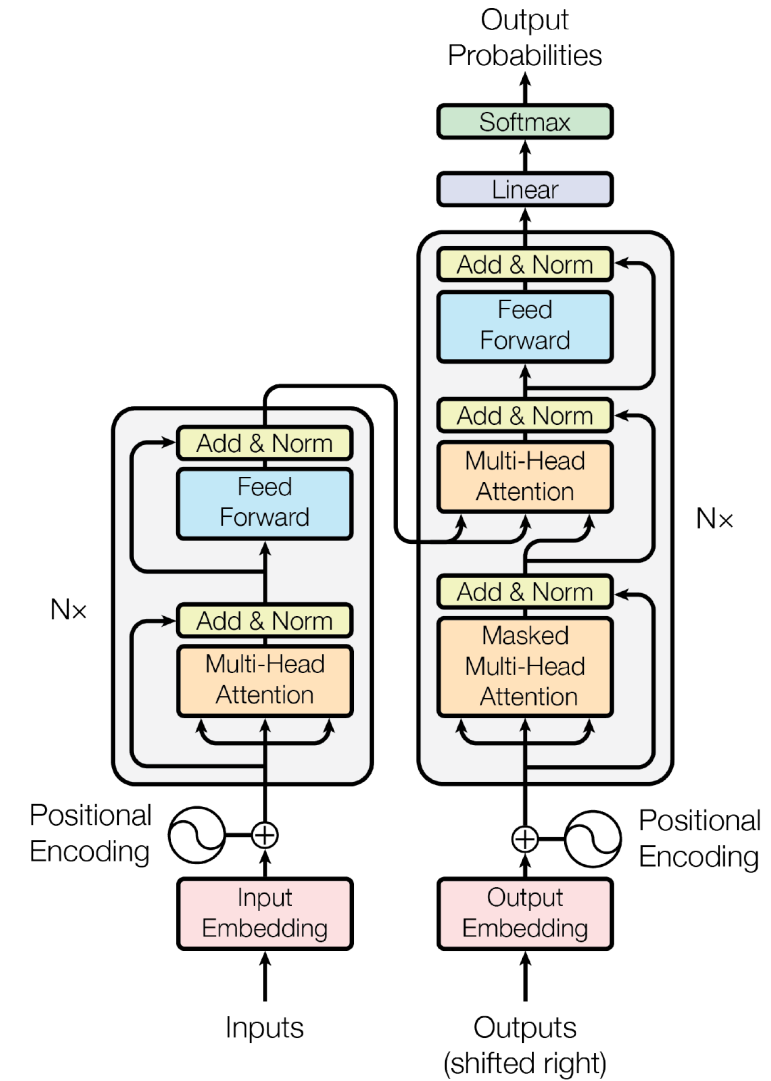
```



Decoder Layer

[7]

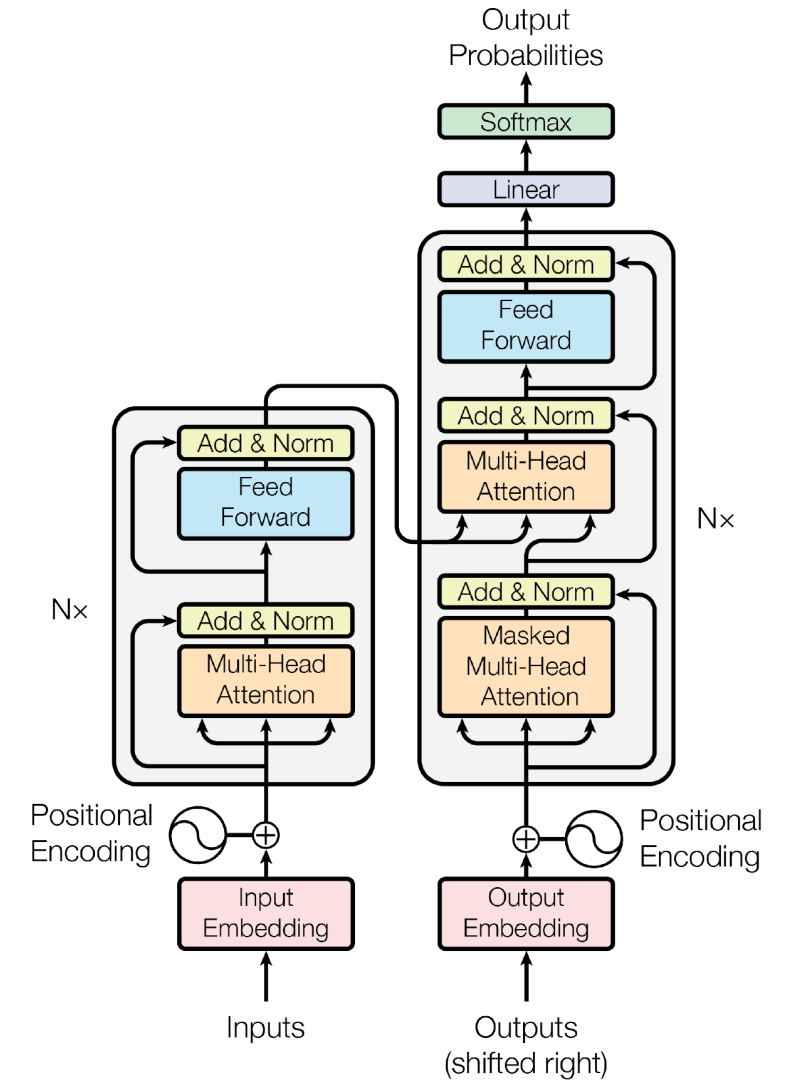
```
def forward(self, x, memory, tgt_mask=None, memory_mask=None):  
  
    # Masked self-attention (decoder attends to previous positions)  
    self_attn_out = self.self_attn(x, x, x, mask=tgt_mask)  
    x = x + self.dropout(self_attn_out)  
    x = self.norm1(x)  
  
    # Encoder-decoder attention (decoder queries, encoder keys/values)  
    cross_attn_out = self.cross_attn(x, memory, memory, mask=memory_mask)  
    x = x + self.dropout(cross_attn_out)  
    x = self.norm2(x)  
  
    # Feed-forward  
    ff_out = self.ff(x)  
    x = x + self.dropout(ff_out)  
    x = self.norm3(x)  
  
    return x
```



Decoder Layer

[7]

```
class DecoderLayer(nn.Module):  
  
    # One decoder layer = masked self-attention +  
    #                               encoder-decoder attention + feed-forward.  
  
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):  
        super().__init__()  
        self.self_attn = MultiHeadAttention(d_model, num_heads, dropout)  
        self.cross_attn = MultiHeadAttention(d_model, num_heads, dropout)  
        self.ff = PositionwiseFeedForward(d_model, d_ff, dropout)  
  
        self.norm1 = nn.LayerNorm(d_model)  
        self.norm2 = nn.LayerNorm(d_model)  
        self.norm3 = nn.LayerNorm(d_model)  
        self.dropout = nn.Dropout(dropout)
```



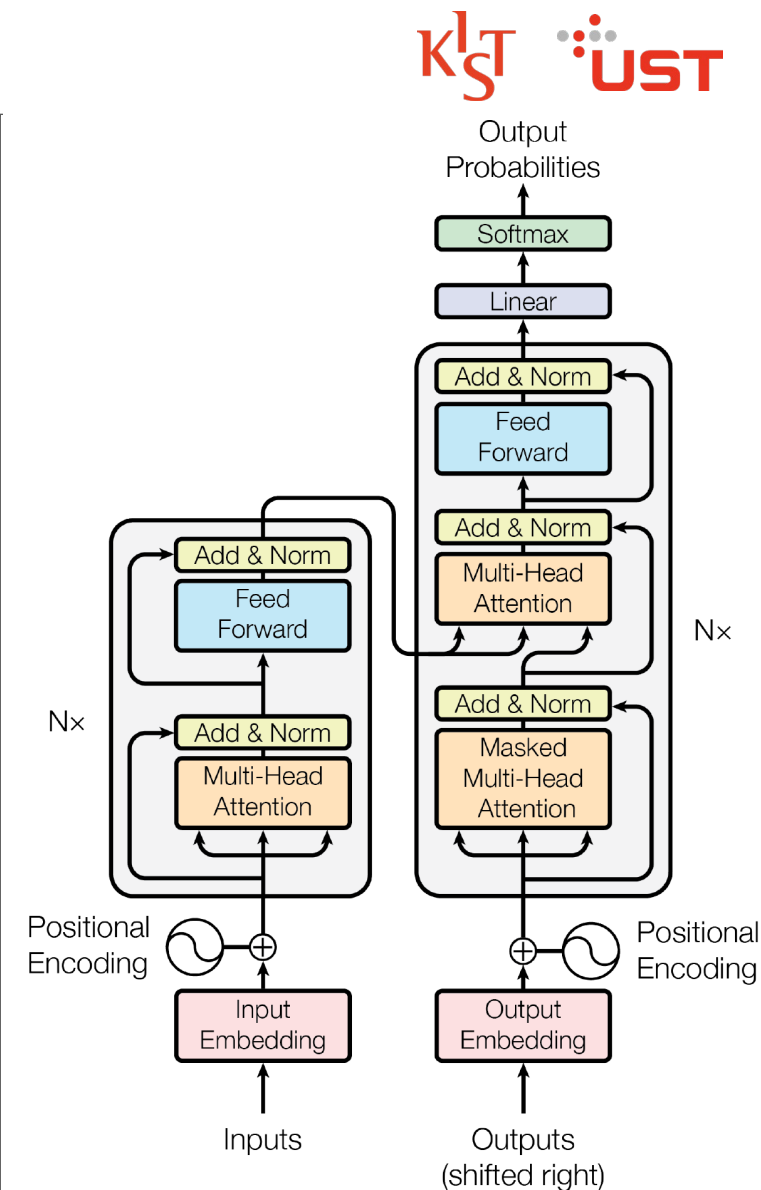
```
class Decoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_layers, num_heads, d_ff,
                  max_len=5000, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos_enc = PositionalEncoding(d_model, max_len)

        self.layers = nn.ModuleList(
            [DecoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)]
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, tgt, memory, tgt_mask=None, memory_mask=None):
        """
        tgt: (batch, tgt_len)
        memory: encoder outputs (batch, src_len, d_model)
        tgt_mask: (batch, 1, tgt_len, tgt_len)
        memory_mask: (batch, 1, 1, src_len)
        """
        x = self.embed(tgt) * math.sqrt(self.d_model)
        x = self.pos_enc(x)
        x = self.dropout(x)

        for layer in self.layers:
            x = layer(x, memory, tgt_mask=tgt_mask, memory_mask=memory_mask)

        return x  # (batch, tgt_len, d_model)
```



Vision Transformer (ViT)

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

**Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}**

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

Vision Transformer (ViT)

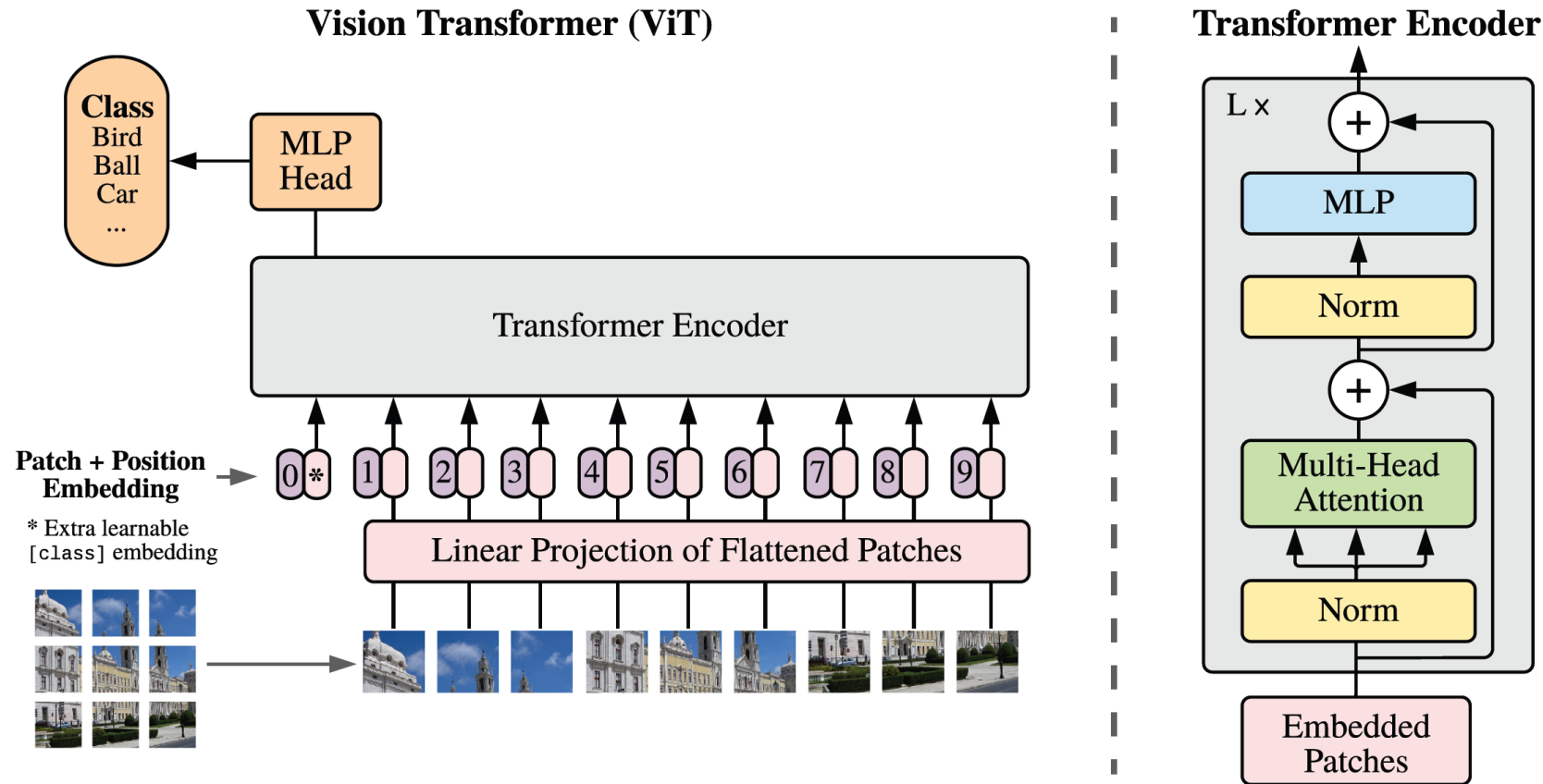
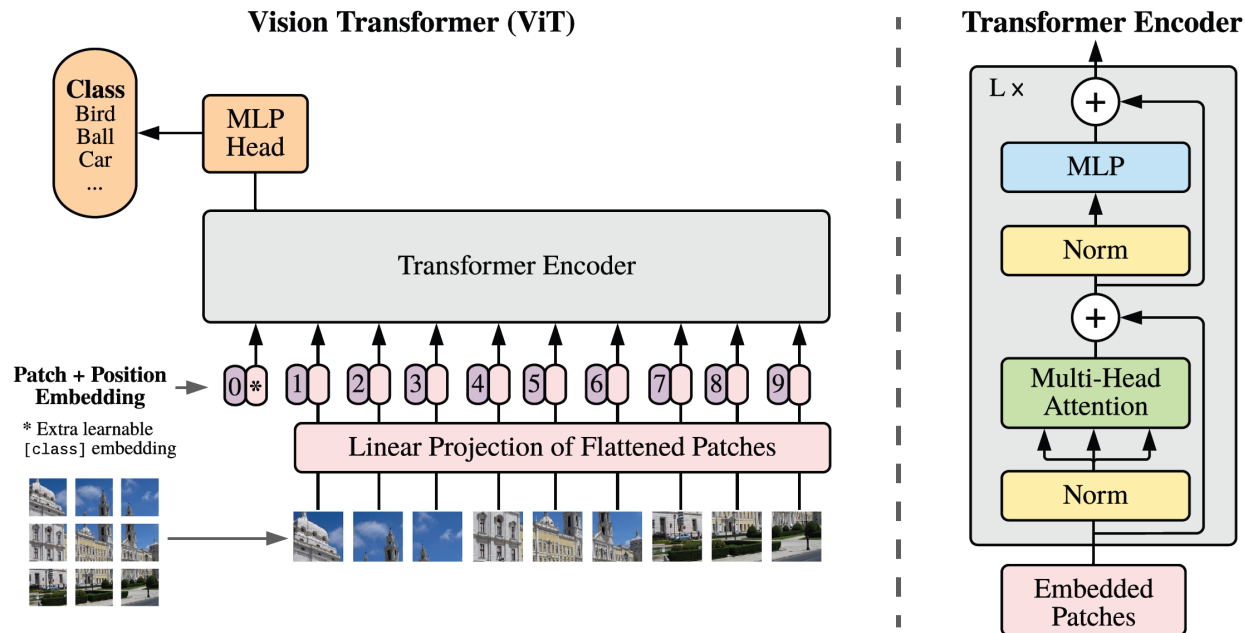


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Vision Transformer (ViT)

The Vision Transformer (ViT) is a deep learning architecture introduced by Dosovitskiy et al. (2020) that applies the Transformer model, originally designed for natural language processing, to image classification. Instead of using convolutional layers, the Vision Transformer treats an image as a sequence of patches and feeds these patches into a standard Transformer encoder.

ViT demonstrated that, when trained on sufficiently large datasets, Transformers can outperform convolutional neural networks (CNNs) on image tasks.



Patch Embedding

Given an input image of size

$$H \times W \times C,$$

ViT divides the image into non-overlapping patches of size

$$P \times P,$$

each containing P^2C pixels.

The number of patches is

$$N = \frac{HW}{P^2}.$$

Each patch is flattened into a vector

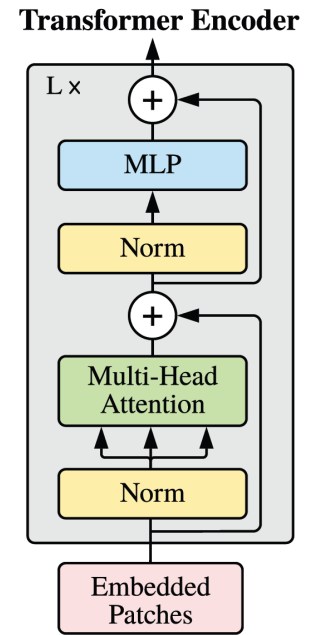
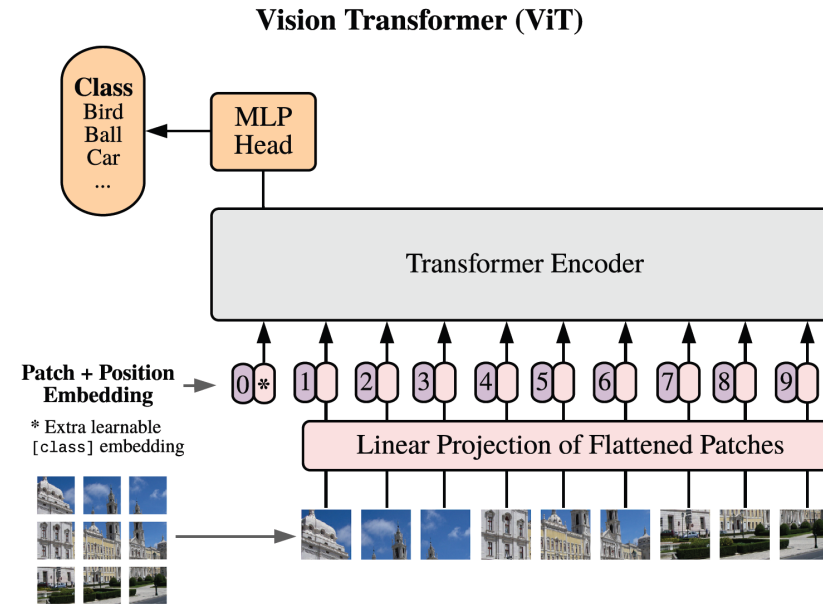
$$x_i \in \mathbb{R}^{P^2C}, \quad i = 1, \dots, N,$$

and then projected into a d -dimensional embedding space using a trainable matrix:

$$z_i = x_i W_E, \quad W_E \in \mathbb{R}^{(P^2C) \times d}.$$

Thus, the image becomes a sequence of tokens:

$$(z_1, z_2, \dots, z_N).$$



Class Token and Positional Embedding

Similar to the [CLS] token in BERT, ViT prepends a learnable class token

$$z_0 \in \mathbb{R}^d$$

to the sequence. This token will be used for classification.

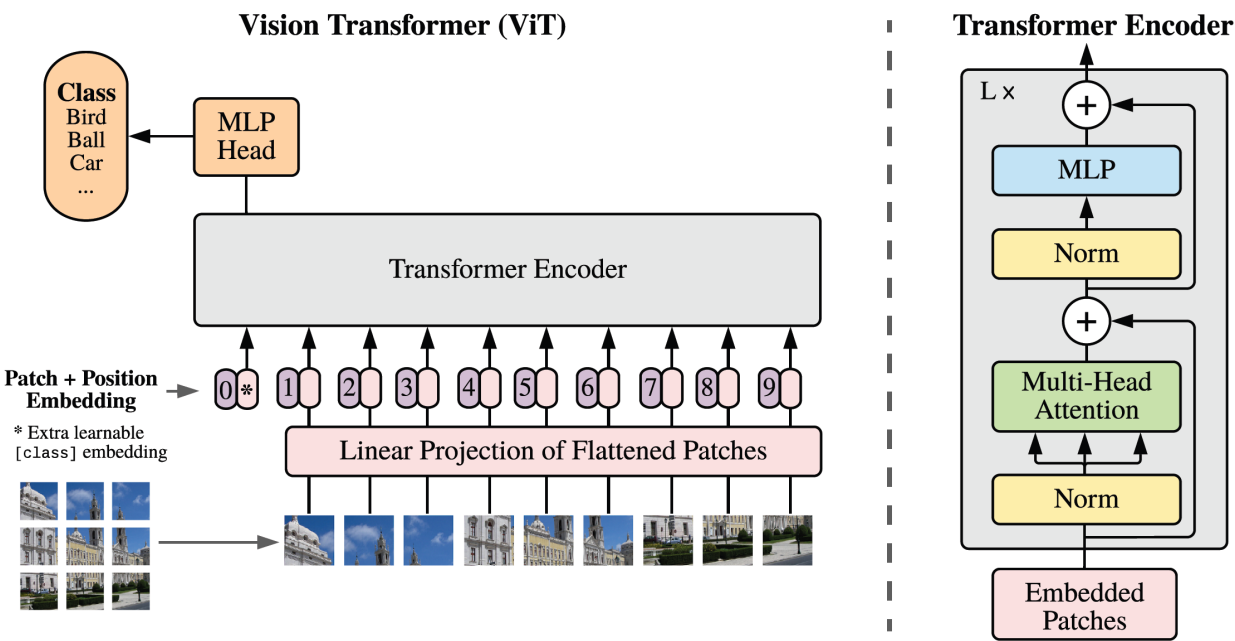
Since Transformers have no inherent positional information, each token receives a positional embedding:

$$\tilde{z}_i = z_i + p_i, \quad p_i \in \mathbb{R}^d,$$

for $i = 0, 1, \dots, N$.

The input to the Transformer encoder is then

$$(\tilde{z}_0, \tilde{z}_1, \dots, \tilde{z}_N).$$



Transformer Encoder

ViT uses the standard Transformer encoder consisting of repeated blocks, where each block contains:

- multi-head self-attention (MHSA),
- a position-wise feed-forward network (FFN),
- residual connections and layer normalization.

A single encoder block transforms the hidden states according to

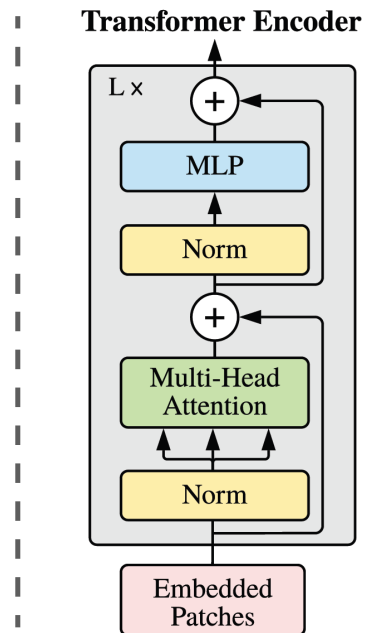
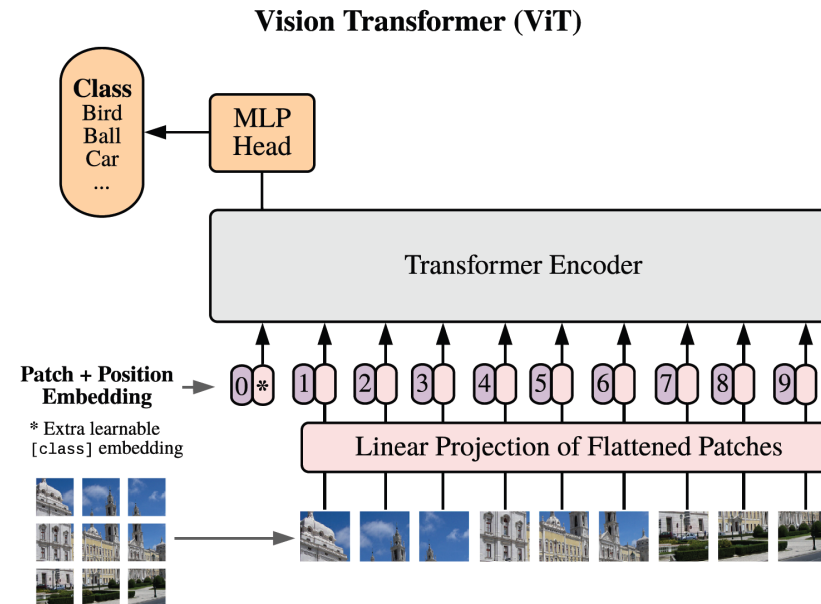
$$\hat{z}^{(\ell)} = \text{MHSA}(\text{LN}(z^{(\ell)})) + z^{(\ell)},$$

$$z^{(\ell+1)} = \text{FFN}(\text{LN}(\hat{z}^{(\ell)})) + \hat{z}^{(\ell)}.$$

After L layers, the hidden state of the class token,

$$z_0^{(L)},$$

serves as the image representation.

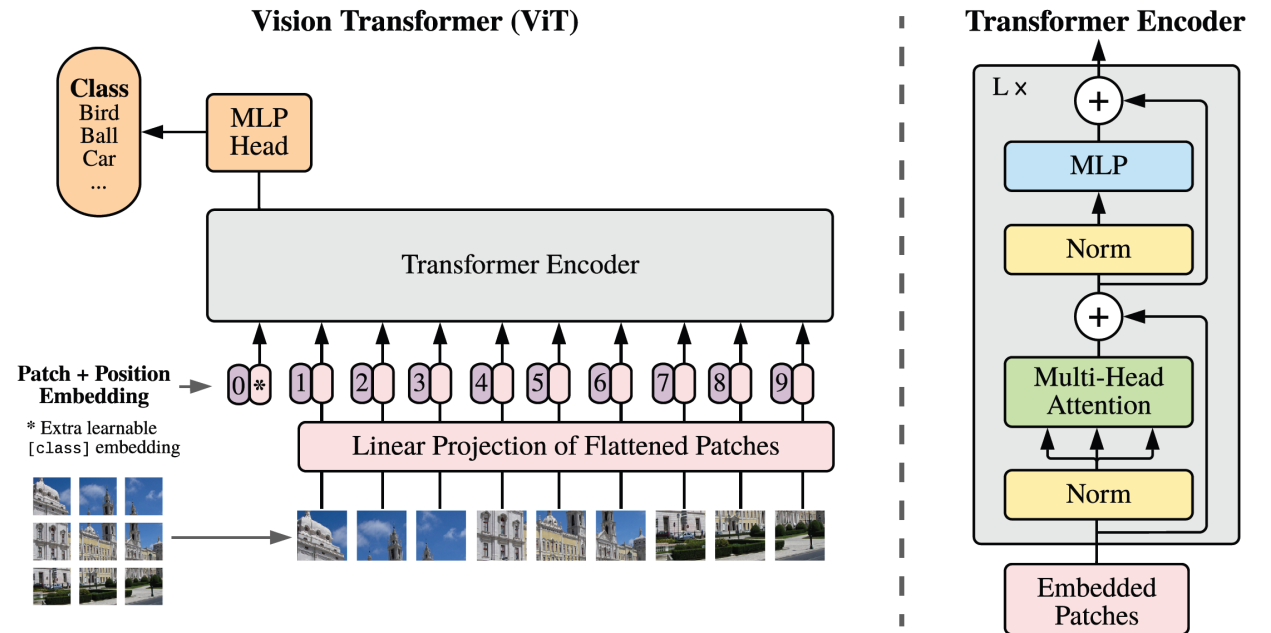


Classification Head

The final class token representation is fed into a linear classifier:

$$y = W_c z_0^{(L)} + b_c,$$

where $W_c \in \mathbb{R}^{K \times d}$ for K output classes.



Why ViT Work

6.1 Global Attention

Unlike CNNs, which have a limited receptive field unless many layers are stacked, ViT computes relationships between *all* patches at every layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

This allows ViT to model long-range dependencies naturally and efficiently.

6.2 Scalability with Data

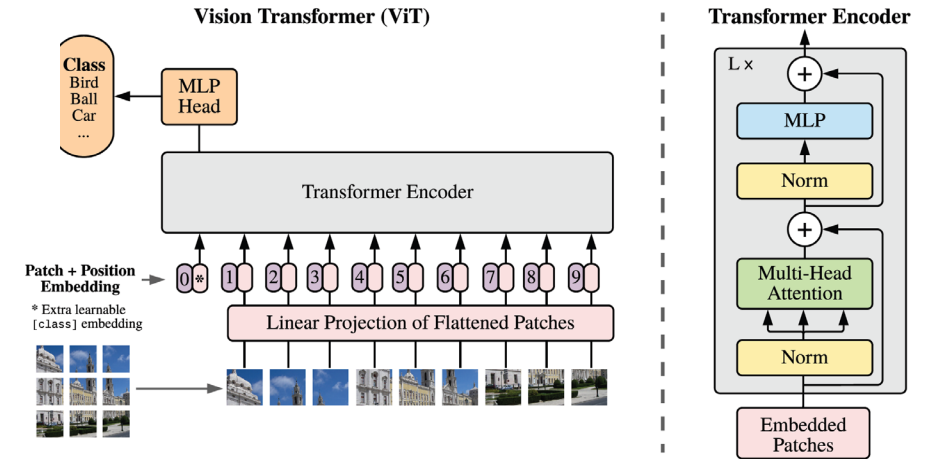
ViT models typically require large-scale training data (e.g. ImageNet-21k, JFT-300M) to outperform CNNs. Once sufficiently trained, they show strong generalization and can be fine-tuned efficiently on downstream tasks.

6.3 Architectural Simplicity

ViT removes inductive biases such as convolutional locality and weight sharing, resulting in a very simple architecture:

- no convolutions,
- no pooling,
- uniform Transformer blocks.

This simplicity makes ViT highly scalable and compatible with hardware accelerators.



SIIM-FISABIO-RSNA COVID-19 Detection

<https://www.kaggle.com/c/siim-covid19-detection>

The screenshot shows the Kaggle competition page for SIIM-FISABIO-RSNA COVID-19 Detection. The left sidebar contains navigation links: Home, Competitions (selected), Datasets, Models, Benchmarks, Game Arena, Code, Discussions, Learn, More, Your Work, and a list of datasets including SIIM-FISABIO-RSNA C..., Chest X-Ray Images (...), wikitext-2-v1, NIH Chest X-rays, and Public API. The main content area has a search bar and a 'Late Submission' button. The competition title 'SIIM-FISABIO-RSNA COVID-19 Detection' is prominently displayed, followed by the description 'Identify and localize COVID-19 abnormalities on chest radiographs'. Below this is a navigation bar with tabs: Overview, Data (selected), Code, Models, Discussion, Leaderboard, Rules, Team, and Submissions. The 'Dataset Description' section explains the task: identifying and localizing COVID-19 abnormalities on chest radiographs. It provides instructions for predicting bounding boxes and classes for findings, and for determining labels within test studies. A code example shows a prediction string: `'Negative for Pneumonia' 'Typical Appearance' 'Indeterminate Appearance' 'Atypical Appearance'`. The 'Dataset information' section states that the train dataset comprises 6,334 chest scans in DICOM format, labeled by radiologists for opacities and overall appearance. It also notes that all images are stored in paths with the form `study / series / image`, where `study` is the study-level prediction ID and `image` is the image-level prediction ID. The hidden test dataset is of roughly the same scale as the training dataset. On the right, a table lists dataset details: Files (7600), Size (128.51 GB), Type (dcm, csv), and License (Subject to Competition Rules).

SIIM-FISABIO-RSNA COVID-19 Detection
Identify and localize COVID-19 abnormalities on chest radiographs

Overview **Data** Code Models Discussion Leaderboard Rules Team Submissions

Dataset Description

In this competition, we are identifying and localizing COVID-19 abnormalities on chest radiographs. This is an object detection and classification problem.

For each test image, you will be predicting a bounding box and class for all findings. If you predict that there are no findings, you should create a prediction of "none 1 0 0 1 1" ("none" is the class ID for no finding, and this provides a one-pixel bounding box with a confidence of 1.0).

Further, for each test study, you should make a determination within the following labels:

'Negative for Pneumonia' 'Typical Appearance' 'Indeterminate Appearance' 'Atypical Appearance'

To make a prediction of one of the above labels, create a prediction string similar to the "none" class above: e.g. `atypical 1 0 0 1 1`

Please see the Evaluation page for more details about formatting predictions.

The images are in DICOM format, which means they contain additional data that might be useful for visualizing and classifying.

Dataset information

The train dataset comprises 6,334 chest scans in DICOM format, which were de-identified to protect patient privacy. All images were labeled by a panel of experienced radiologists for the presence of opacities as well as overall appearance.

Note that all images are stored in paths with the form `study / series / image`. The `study` ID here relates directly to the study-level predictions, and the `image` ID is the ID used for image-level predictions.

The hidden test dataset is of roughly the same scale as the training dataset.

Files	7600 files
Size	128.51 GB
Type	dcm, csv
License	Subject to Competition Rules

How to download the dataset, and **explanations** of the training code and inference code will be uploaded to the class website as a **separate** PDF file.

The training code is based on ViT learned in our class and will be presented on the **final exam**.

Thank you!

